

SIMATS ENGINEERING
Department of Computer Science &
Engineering ASSESSMENT TOOL 1- ANALYTICAL
PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Arun Pandian K	Reg.No.:	192511086
Department:	CSE	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Ans. Problem Understanding

The processor performs arithmetic on signed 8-bit numbers using 2's complement representation.

Given:

- +45
- -23

Step 1: Convert Decimal to Binary

+45

45 = 00101101

+23

23 = 00010111

-23 (2's Complement)

1's Complement:

11101000

Add 1:

11101001

Therefore,

- **+45 = 00101101**
- **-23 = 11101001**

Addition (+45 + (-23))

00101101

+ 11101001

00010110

Discard carry.

Result:

00010110 = +22

Overflow Check

Operands have different signs.

Positive + Negative = **No Overflow**

Subtraction (+45 – (–23))

Subtracting a negative means adding a positive.

$$+45 = 00101101$$

$$+23 = 00010111$$

$$00101101$$

$$+00010111$$

$$01000100$$

Result

$$01000100 = +68$$

Overflow:

Both operands are positive and result is positive.

No Overflow

Processor Overflow Detection

Overflow occurs when

- Positive + Positive → Negative
- Negative + Negative → Positive

Here,

Addition:

$$+45 + (-23) = +22$$

No overflow.

Subtraction:

$$+45 - (-23) = +68$$

No overflow.

Final Results

Addition = **+22**

Subtraction = **+68**

Overflow = **Not Occurred**

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

ANS. 4-bit Carry Look Ahead Adder (CLA)

Problem Understanding

A 4-bit Carry Look Ahead Adder (CLA) is used to reduce the delay caused by carry propagation in a Ripple Carry Adder (RCA). It generates carry signals in advance using **Generate (G)** and **Propagate (P)** signals.

Given Inputs

A = 1011

B = 0110

C_{in} = 0

Step 1: Generate (G) and Propagate (P) Signals

Formula:

$$G_i = A_i \times B_i$$

$$P_i = A_i \oplus B_i$$

Bit	A	B	G	P
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Step 2: Carry Computation

Carry equations:

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0$$

Since **C_{in} = 0**

$$C1 = 0$$

$$C2 = 1$$

$$C3 = 1$$

$$C4 = 1$$

Step 3: Sum Calculation

Formula

$$Si = Pi \oplus Ci$$

Bit	P	Carry	Sum
0	1	0	1
1	0	0	0

Bit	P	Carry	Sum
-----	---	-------	-----

2	1	1	0
---	---	---	---

3	1	1	0
---	---	---	---

Result

10001_2

Decimal

$11 + 6 = 17$

Comparison

Ripple Carry Adder

Carry propagates one stage at a time

Higher delay

Simple design

Slower

Carry Look Ahead Adder

Carries computed simultaneously

Very low delay

Slightly complex

Faster

Conclusion

CLA improves processor speed because all carry bits are generated simultaneously. Therefore, it is widely used in modern high-speed processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Ans. Problem Understanding

Booth's Algorithm is used to multiply signed binary numbers efficiently using 2's

complement representation. It reduces the number of addition and subtraction operations by examining adjacent bits of the multiplier.

Step 1: Binary Representation

Multiplicand (M) = -6 = 1010

Multiplier (Q) = +5 = 0101

Accumulator (A) = 0000

Booth Bit (Q-1) = 0

Number of bits = 4

Step 2: Booth's Rules

- **10** → **A = A - M**
- **01** → **A = A + M**
- **00 or 11** → **No operation**
- Perform **Arithmetic Right Shift (ARS)** after each step.

Step 3: Iterations

Step	Q ₀ Q ₋₁	Operation	After Shift (A Q Q-1)
1	10	A = A - M	0011 0010 1
2	01	A = A + M	1110 1001 0
3	10	A = A - M	0010 0100 1
4	01	A = A + M	1110 0010 0

Final Product

AQ = 11100010₂

Decimal Result

-30

Advantages of Booth's Algorithm

- Handles positive and negative numbers automatically.
- Reduces the number of addition and subtraction operations.
- Faster than conventional binary multiplication.
- Widely used in modern processors.

Conclusion

The multiplication of $-6 \times +5$ gives -30 , and Booth's Algorithm performs the operation efficiently using arithmetic shifts and fewer arithmetic operations.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Ans. Problem Understanding

The processor performs binary division using **Restoring Division** and **Non-Restoring Division** algorithms. The dividend is **13** and the divisor is **3**.

Step 1: Binary Representation

Dividend (Q) = $13 = 1101_2$

Divisor (M) = $3 = 0011_2$

Accumulator (A) = 0000

Restoring Division Method

Algorithm

1. Left shift (A,Q).
2. Subtract divisor ($A = A - M$).
3. If A is negative, restore A by adding M and set quotient bit = 0.
4. If A is positive, set quotient bit = 1.

5. Repeat for 4 cycles.

Iteration Table

Step	Operation	Quotient
1	Subtract → Negative → Restore	0
2	Subtract → Positive	1
3	Subtract → Negative → Restore	0
4	Subtract → Positive	1

Final Result

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Non-Restoring Division Method

Algorithm

1. Left shift (A,Q).
2. If previous remainder is positive, subtract divisor.
3. If previous remainder is negative, add divisor.
4. No restoring operation is required.
5. Repeat until all bits are processed.

Iteration Table

Step	Operation	Quotient
1	Subtract	0
2	Add	1
3	Subtract	0
4	Add	1

Final Result

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Comparison

Restoring Division

Restores negative remainder

More arithmetic operations

Slower

Higher complexity

Non-Restoring Division

No restoring operation

Fewer arithmetic operations

Faster

More efficient

Conclusion

Both algorithms produce:

- **Quotient = 4**
- **Remainder = 1**

Non-restoring division is preferred because it eliminates unnecessary restoring steps, reducing execution time in modern processors.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Ans. Problem Understanding

IEEE 754 is the standard used to represent floating-point numbers in computers. It divides the number into **Sign**, **Exponent**, and **Mantissa (Fraction)** fields.

Given decimal number:

-13.25

Step 1: Convert to Binary

Integer part:

$$13 = 1101_2$$

Fractional part:

$$0.25 = 0.01_2$$

Therefore,

$$13.25 = 1101.01_2$$

Normalized form:

$$1.10101 \times 2$$

Single Precision (32-bit)

- **Sign bit** = 1 (negative number)
- **Exponent** = $3 + 127 = 130 = 10000010_2$
- **Mantissa** = 101010000000000000000000

Field	Value
Sign	1
Exponent	10000010
Mantissa	101010000000000000000000

Double Precision (64-bit)

- **Sign bit** = 1
- **Exponent** = $3 + 1023 = 1026 = 10000000010_2$
- **Mantissa** = 10101000000000000000000000000000000000000000

Code of Conduct:

I Arun Pandian K (192511086) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Arun Pandian K

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation